Interim: Emulating Lua Execution through different Recurrent Neural Network Architectures: A Comparative Study in Example-Based Learning.

Richard Coric

University of Lincoln, School of Engineering and Physical Sciences
26414124@students.lincoln.ac.uk

Keywords: Deep Learning · Compiler Development · Long Short-Term Memory (LSTM) · Recurrent Neural Network (RNN) · Gated Recurrent Unit (GRU).

1 Literature Review

1.1 Introduction

Prior to transformer models (Vaswani et al., 2017), Recurrent Neural Networks (RNNs) were widely used for natural language processing (NLP) due to their ability to process sequential data and their capability to learn longer-term dependencies than Feed-Forward Networks.

There have been many improvements to RNNs, S. Hochreiter and J. Schmidhuber. (1997) addressed the issue of vanishing gradients and developed the Long-Short Term Memory (LSTM) model, which can keep track of longer-term dependencies via a gating mechanism. Later, the GRU (Gated Recurrent Unit) was introduced as a simpler LSTM architecture in an Encoder-Decoder RNN for Machine Translation.

The transformer model has been the standard for NLP (see Appendix 3.1) tasks (Patwardhan et al., 2023) ever since its introduction in 2017 (Vaswani et al., 2017). RNNs continue to demonstrate usage in specific domains and can reach similar outcomes to a transformer model with the added benefit of parallelisability (Feng et al., 2024) by updating the architecture to remove hidden states, adding a normalisation element (this is illustrated in figure 1) thus removing the need for BPTT (backpropagation through time) allowing the use of parallel algorithms such as parallel prefix-scan (Blelloch, 1990).

1.2 RNNs in Code-Related Tasks

Beyond NLP, RNNs have been used for compiler-like tasks since 2014 with the Neural Turing Machine (Graves et al., 2014). Researchers employed various architectures to achieve their objectives. Common models included standard RNNs, LSTMs, or a combination to create an encoder-decoder structure (Burgueño et al., 2021; Graves et al., 2014; Priya et al., 2017-12; Rahman et al., 2020; Zaremba et al., 2022).

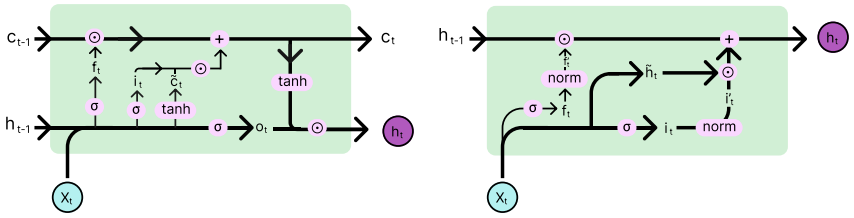


Fig. 1: LSTM Architecture (Left) and MinLSTM (Right)

Methodologies Table 1 summarises research in code-related tasks using RNNs. It gives an overview of the methods used by other researchers.

Table 1: Summary of RNNs in Code-Related Tasks

Author	Dataset	Method	Metric	Results
Graves. et al. (2014)	A mix of data of simple algorithms.	RNN with external memory, supervised learning, 3 layer.	Bits-per-sequence	Able to generate code, dataset size had no impact.
Wojciech and Sutskever (2015)	Set of custom made, Python like Programs.	LSTM, and curriculum learning.	Accuracy	99% accuracy for addition, LSTM learned programs.
Reed. et al. (2015)	A set of programs custom made.	NPI - an LSTM based network with arguments and Supervised Learning. Also used Curriculum Learning.	Ability to generalise and per-sequence accuracy	Was able to learn to execute complex programs, <90% accuracy in most cases.
Wojciech. et al. (2016)	Short python programs.	RNN controller: LSTM and GRU, Reinforcement Learning	Accuracy.	Q-Learning failed on most tasks, but succeeded with Watkins and Dynamic Count.
Balog. et al. (2017)	Custom made DSL.	Encoder Feed Forward, Decoder RNN.	Time to find a program.	DeepCoder showed performance speed ups up to 475×

Priya R. et al. (2017)	Open Source from Github (Python, Java, C#).	Character-based RNN 512 hidden state.	Loss and Accuracy.	Performed well, sometimes generated code with errors, RNN were effective.
Rahman. et al. (2020)	Code from Oj competitive programming system.	LSTM and Attention.	Precision, Recall, F1.	Attention improved accuracy from 31% to 62% and approx. 0.9 F1
Laich L. et al. (2020)	Rico dataset of PlayStore apps.	MLP, CNN and RNN.	Existing Research and Accuracy	Improved from 35% to 71% accuracy.
Burgueño. et al. (2021)	Custom Dataset using GANs.	LSTM + Attention, with Dropout and Regularisation.	Accuracy and training time.	0.94 accuracy after increasing dataset size, this was also proportional to train time.

Past research looked at two main approaches to code prediction: Character-based RNNs (Priya et al., 2017-12), where the model predicts the next character from a probability distribution. Word-based models, in contrast, have full word tokens and predict the next complete word. Results show that word-based RNNs achieve a higher semantic accuracy (Burgueño et al., 2021) than character-based RNNs (Priya et al., 2017-12). Supported by these findings, this research implements word-level RNNs in order to attempt to generate more error-free code. Specifically, our model aims to predict program output from a given input without directly executing the code.

Evaluation Metrics Researchers commonly use accuracy to evaluate their models (Burgueño et al., 2021; Priya et al., 2017-12; Zaremba and Sutskever, 2014; Zaremba et al., 2022). Accuracy tells us the fraction of correct examples but lacks any notion of semantic correctness; this can be an issue for code generation because there are often multiple correct solutions. Better methods involve using BLEU to measure the token-level similarity (Burgueño et al., 2021), providing additional insight into the performance.

Researchers used both execution-based and functional correctness metrics. Balog M. computed the time taken to find a program – however, most struggle to evaluate for functional correctness. For instance, Laich L. et al. used human feedback to find how many corrections a user would need to make for the program to be correct.

This research aims to use a balance of metrics to give an overview of how well the model performs. This will include Loss, Perplexity, BLEU and Sentence Similarity.

Limitations Current research has a few limitations; one limitation is that studies reported difficulty producing compilable and error-free code (Priya et al., 2017). They found that this did not improve with more data. One way this

research addresses this is Syntax-aware training, which involves predicting the next token based on a set of grammar rules and only allowing valid tokens. This can be likened to how a compiler checks for syntax errors.

Other researchers had issues with the dataset size and quality. Burgueño et al. (2021) found that the dataset bottlenecks the model. They proposed using GANs to generate Synthetic data. However, Balog M. et al. (2017) found that synthesised programs may sometimes be too simplistic and fail to address real-world applications. Other researchers had issues with the dataset size and quality (Burgueño et al., 2021, Rahman et al., 2020, Reed and De Freitas, 2015). This research aims to use Open Source LLMs for Data Augmentation to generate synthetic data, which will be used to train the model.

Another way current models are limited is that they fail to understand the Semantic Meaning of code, which affects any error detection or prediction tasks (Rahman et al., 2020). To address this, this research aims to use Semantic Embeddings to encode the program's meaning or the AST (Abstract Syntax Tree). This will allow the model to understand the meaning of the code and not just the syntax.

RNNs also tend to fail to generalise for longer inputs (Reed and De Freitas, 2015) or follow a program specification too closely (Laich et al., 2020). This can also lead to the model memorising the dataset rather than learning the underlying patterns (Zaremba and Sutskever, 2014). This research aims to address this by using Curriculum Learning, which involves training the model on simpler tasks first and gradually increasing the complexity of the tasks. Curriculum learning has been shown to improve generalisation and learning speed (Reed and De Freitas, 2015; Zaremba and Sutskever, 2014).

1.3 Transformer Models

The transformer architecture is an improvement upon sequencing models like RNNs. RNN-based models have been shown to struggle with long-distance dependencies and are inherently difficult to parallelise. The transformer consists of an encoder and a decoder block. The encoder processes the input sequence, and the decoder generates the output sequence. The transformer model uses self-attention mechanisms to learn the dependencies between words in a sentence. This allows the model to learn long-range dependencies more effectively than RNNs.

Additionally, the transformer model is highly parallelisable because the model does not process the input in sequence but all at once.

More recently, transformer models have been used for code generation, completion, and summarisation tasks. These are summarised in Table 2. Most authors fine-tune pre-trained models like GPT3 for specific tasks or use existing models as a base. This was shown to be effective in most cases, where most re-

Table 2: Summary of Transformer Models in Code-Related Tasks

Author	Dataset	Method	Metric	Results
Bunel. et al (2016)	Generated I/O Examples for Swap Increment Addition and Sort	Neural Network ANC	correctness, halting, efficiency, and confidence, weighted sum to form the total loss function	adapts to simple generic algorithms
Chen. et al. (2021)	Python GitHub Repositories	Temperature, Transformer	pass@k - number of samples that pass per problem, Human Eval.	improved pass rate from 28.8% to 70.2% on the HumanEval dataset.
D. O. Bui. et al. (2023)	HumanEval, CodeXGLUE, Human Input	GPT Transformer	pass@k CodeBLEU	a library which makes it easy to manipulate code models.
Cummins. et al. (2023)	A suite of benchmark datasets of C/C++ code examples	Transformer BPE	BLEU and Success Rate.	90.5% success rate 0.952 BLEU
Munley. et al. (2023)	OpenACC and their own Dataset	GPT, DeepSeek, CodeLlama + RAG	GPU Time Taken, and Pass/Fail Rates.	Tested various models, most passed the majority of tasks.
Gu. Qiuhan (2023)	Processing Go Code using Tree Sitter	CodeT5, Transformer	Code coverage and the percentage of syntax error and undefined behaviour	3.38% coverage, 2.79% syntax errors, 0% of undefined behaviour.
Kim. et al. (2023)	HotpotQA, Movie Recommendation, ParallelQA and other benchmark tasks.	Transformers, GPT	Accuracy and Latency, Success Rate	latency speed up 3.7x, cost savings of up to 6.7x, and accuracy improvement of up to ∼9%
Li, L. et al. (2024)	MAmmoTH model, 18,000 examples from GSM8K, 3,000 from NumGLUE, and 15,000 from MATH.	Attention, CoT, PoT, Reinforcement Learning, Transformer, Llama and GPT	Base-Models and out-of-domain datasets (one that differs a lot from the training data).	Improvement of 6.5% on Llama-Base model, 4.3% on MistralBase model across 8 mathematical calculation datasets.
Grubisic. et al. (2024)	LLVM IR (Intermediate Representation).	Llama - Transformer	Track % improvement over Random Sampling, Greedy Decoding, and Nucleus	2.87% to 5% improvement.
Hu. et al. (2024)	Sintel movie dataset and Custom Made dataset.	Multimodal Learning, GPT-V (GPT Vision).	CLIP Similarity Score (Text-to-Image metric), Human Evaluation	5.6 CLIP on BlenderGPT baseline, and 88.9 on Scene Craft
Chae. et al. (2024)	Big-Bench Hard Reasoning Tasks.	GPT3 and CodeLlama, Zero-Shot CoT.	Improvement over Baseline Models	11% improvement over GPT3.5-Turbo
Cummins. et al. (2024)	CodeLlama LLVM IR. MiBench Benchmark	CodeLlama as a Base.	Vary for tasks: Perplexity, BLEU, Success Rate, pass@k	Outperforms in 61% of cases, significant improvement over baseline models.

search showed a significant improvement compared to baseline models. Comparing against baseline models allowed past research to put results into perspective.

Limitations Transformer-based approaches tended to suffer similar limitations to RNN-based approaches. Researchers found that pre-trained models are biased towards certain languages or frameworks when trained on various languages (Bui et al., 2023; Chen et al., 2021). This research only uses Lua examples and thus should not suffer from this level of bias. However, it should be noted that the model may favour certain implementations over others; to mitigate this, the model will be trained on a diverse dataset.

Additionally, models struggled with processing large input sequences (Cummins et al., 2023) and solved it using a restricted input; however, as proposed in section 1.2, curriculum learning is a viable approach to train the model on increasing input sizes. As well as input size, the models produced are algorithmically complex and sequential (Grubisic et al., 2024), making it impossible to train in parallel.

The use of MinLSTM and MinGRU will allow for longer sequences and faster training due to their simplified and, thus, parallelisable nature (Feng et al., 2024).

2 Progress Update

Based upon objectives outlined in the proposal, the following progress has been made: A Literature Review has been conducted, which gave an insight into the current state of using language models for various code-related tasks.

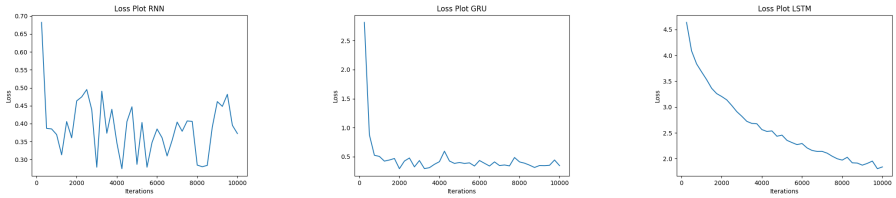
RNN, GRU, and LSTM models were also implemented into a small language model to gauge their performance. The software was implemented using the PyTorch library, which provides a simple interface for constructing models. PyTorch was proven to be a good choice for the task as it provides simple GPU access, greatly speeding up the training process. The structure of the software is shown in Figure 3.

2.1 Preliminary Results

The three models were trained on a small corpus taken from "Alice in Wonderland"; this is a small dataset which gave an insight into how these models perform.

The models were trained for $10,000$ epochs with a 6×10^{-3} learning rate. The Negative Log Likelihood Loss was plotted for every 250th epoch, as seen in Figure 2.

The results indicate the GRU model (2b) converges the fastest; however, it was trained for too long, as the loss remained relatively stable after $2,000$ epochs. Similar results could have been achieved with only $2,000$ epochs. A Standard RNN model (2a) also converges quickly; however, it exhibits significant loss oscillations compared to the other models. This instability is likely due to vanishing gradients or the RNN struggling to learn long-term dependencies.



(a) RNN Loss (b) GRU Loss (c) LSTM Loss

Fig. 2: Loss Curves for RNN, LSTM and GRU Models

In contrast, the LSTM model (2c) converges the slowest but demonstrates the highest stability during training. Although the LSTM performed the worst in loss, this outcome may be due to improper hyperparameters since no tuning was done.

2.2 Changes to Objectives

After reviewing the current literature, some changes will be made to the current objectives. Instead of learned embeddings, this research will use Byte-Pair Encoding and Semantic Embeddings. Not only will Lua code data be collected, but it will also be generated using some form of existing generative AI such as GPT3.

As part of the evaluation strategy, this research will additionally use a baseline model such as CodeLlama (Rozière et al., 2024), StarCoder (Li et al., 2023) to give an idea of how well this research compares to existing models.

Finally, a range of metrics, in addition to Perplexity and Cross-Entropy, will be used to evaluate the model, including Sentence Similarity and Pass@k.

2.3 Supervisory Engagement

Supervisory meetings took place every one to two weeks, where we discussed progress and plans for the rest of this research. Based on the time plan proposed in Figure 4 there are four tasks which were left uncompleted due to other commitments. In order to improve the time management on this research, there will be a more consistent time block where the research will be carried out per week.

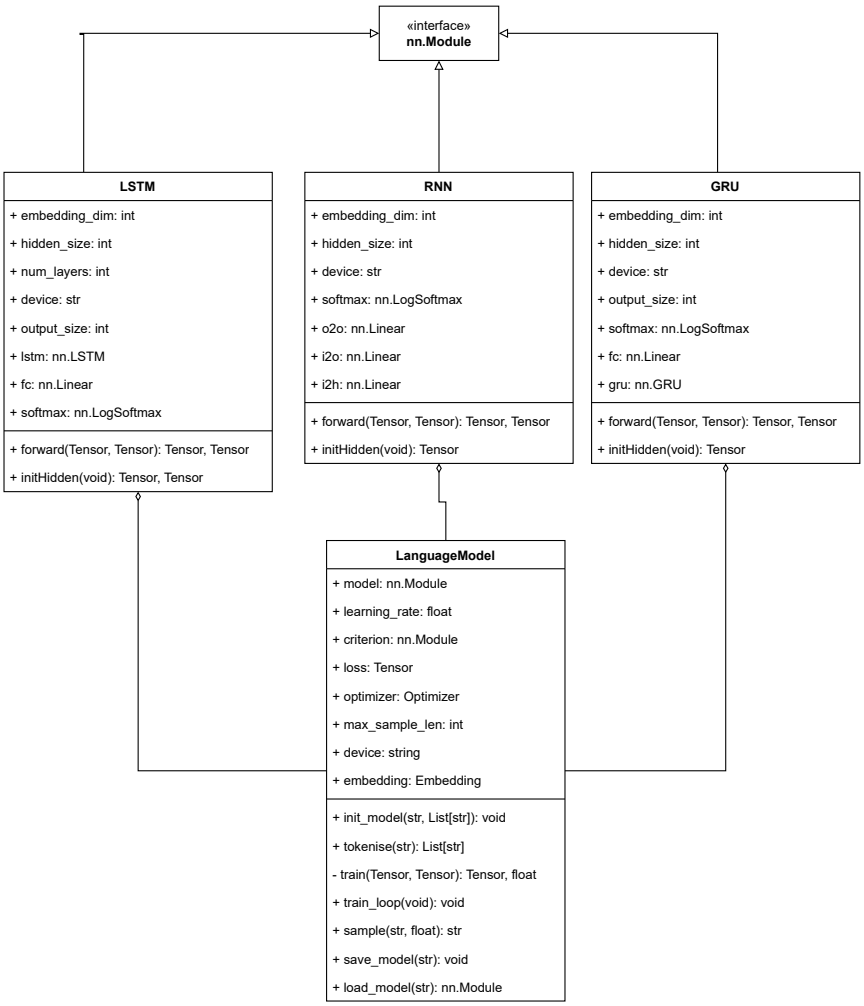


Fig. 3: Class Diagram showing current Implementation

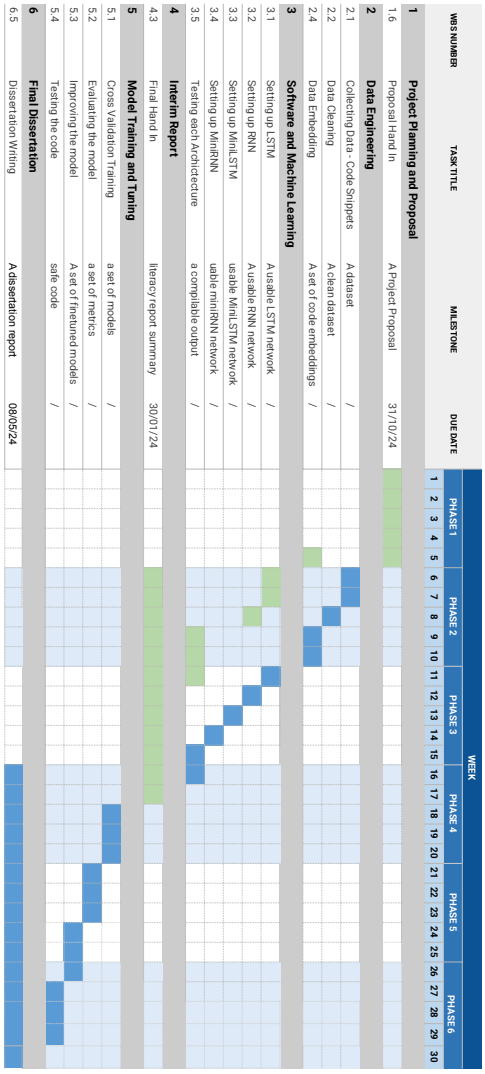


Fig. 4: Updated Gantt Chart with current tasks completed (green)

3 Appendix

3.1 Acronyms

ANC	Adaptive Neural Compilation
AST	Abstract Syntax Tree
BLEU	Bilingual Evaluation Understudy
BPTT	Backpropagation Through Time
CoT	Chain of Thought
CNN	Convolutional Neural Network
DSL	Domain Specific Language
GAN	Generative Adversarial Network
GPT	Generative Pre-trained Transformer
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit
LSTM	Long Short-Term Memory
MinLSTM	Minimal Long Short-Term Memory
MLP	Multi-Layer Perceptron
NLP	Natural Language Processing
NPI	Neural Programmer-Interpreters
PoT	Program of Thought
RAG	Retrieval Augmented Generation
RNN	Recurrent Neural Network

References

1. Balog, M., Gaunt, A.L., Brockschmidt, M., Nowozin, S. and Tarlow, D. (2017-03-08) DeepCoder: Learning to Write Programs. In: Anonymous Proceedings of ICLR'17 Microsoft.
2. Blelloch, G.E. (1990) Prefix Sums and Their Applications.
3. Bui, N.D.Q., Le, H., Wang, Y., Li, J., Gotmare, D. and Hoi, S.C.H. (2023) CODETF: ONE-STOP TRANSFORMER LIBRARY FOR STATE-OF-THE-ART CODE LLM. Available from https://arxiv.org/abs/2306.00029.
4. Bunel, R., Desmaison, A., Kohli, P., Torr, P.H.S. and Pawan Kumar, M. (2016) Adaptive Neural Compilation. Available from https://arxiv.org/abs/1605.07969.
5. Burgueño, L., Cabot, J., Li, S. and Gérard, S. (2021) A generic LSTM neural network architecture to infer heterogeneous model transformations. Software and Systems Modeling, 21(1) 139..
6. Chae, H., Kim, Y., Kim, S., Ong, K.T., Kwak, B., Kim, M., Kim, S., Kwon, T., Chung, J., Yu, Y. and Yeo, J. (2024) Language Models as Compilers: Simulating Pseudocode Execution Improves Algorithmic Reasoning in Language Models. In: Anonymous Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing Association for Computational Linguistics. 22471–22502 Available from https://aclanthology.org/2024.emnlp-main.1253/.
7. Chen, M., Tworek, J., Jun, H., Yuan, Q., Ponde De Oliveira Pinto, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F.P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W.H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A.N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., Mcgrew, B., Amodei, D., Mccandlish, S., Sutskever, I. and Zaremba, W. (2021) Evaluating Large Language Models Trained on Code. Available from https://arxiv.org/abs/2107.03374.
8. Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H. and Bengio, Y. (2014) Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation..
9. Cummins, C., Grubisic, D., Elhoushi, M., Roziere, B., Gehring, J., Gloeckle, F., Hazelwood, K., Synnaeve, G., Leather, H., Ai, M. and Liang, Y. (2023) Large Language Models for Compiler Optimization. Available from https://arxiv.org/abs/2309.07062.
10. Cummins, C., Grubisic, D., Rozière, B., Gehring, J., Synnaeve, G., Leather, H. and Ai, M. (2024) Meta Large Language Model Compiler: Foundation Models of Compiler Optimization. Available from https://ai.meta.com/research/publications/meta-large-language-model-compiler-foundation-models-of-compiler-optimization/.
11. Feng, L., Tung, F., Ahmed, M.O., Bengio, Y. and Hajimirsadegh, H. (2024) Were RNNs All We Needed? Available from http://arxiv.org/abs/2410.01201[accessed Oct 20, 2024].
12. Graves, A., Wayne, G., Deepmind, G. and London. (2014) Neural Turing Machines. Available from https://arxiv.org/abs/1410.5401.
13. Grubisic, D., Cummins, C., Ai, M., Seeker, V. and Leather, H. (2024) Priority Sampling of Large Language Models for Compilers. In: Anonymous EuroMLSys '24:

Proceedings of the 4th Workshop on Machine Learning and Systems Association for Computing Machinery. 91–97.
14. Gu, Q. and Wang, Y. (2023) LLM-Based Code Generation Method for Golang Compiler Testing. In: Anonymous ACM..
15. Kim, S., Moon, S., Tabrizi, R., Lee, N., Mahoney, M.W., Keutzer, K., Gholami, A. and Derrickson, S. (2025) An LLM Compiler for Parallel Function Calling. In: Anonymous Proceedings of the 41st International Conference on Machine Learning JMLR.org. 24370–2439.
16. Laich, L., Bielik, P. and Vechev, M. (2020) Guiding Program Synthesis by Learning to Generate Examples. In: Anonymous International Conference on Learning Representations Available from https://openreview.net/forum?id=BJl07ySKvS.
17. Li, R., Allal, L.B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J., Liu, Q., Zheltonozhskii, E., Zhuo, T.Y., Wang, T., Dehaene, O., Davaadorj, M., Lamy-Poirier, J., Monteiro, J., Shliazhko, O., Gontier, N., Meade, N., Zebaze, A., Yee, M., Umapathi, L.K., Zhu, J., Lipkin, B., Oblokulov, M., Wang, Z., Murthy, R., Stillerman, J., Patel, S.S., Abulkhanov, D., Zocca, M., Dey, M., Zhang, Z., Fahmy, N., Bhattacharyya, U., Yu, W., Singh, S., Luccioni, S., Villegas, P., Kunakov, M., Zhdanov, F., Romero, M., Lee, T., Timor, N., Ding, J., Schlesinger, C., Schoelkopf, H., Ebert, J., Dao, T., Mishra, M., Gu, A., Robinson, J., Anderson, C.J., Dolan-Gavitt, B., Contractor, D., Reddy, S., Fried, D., Bahdanau, D., Jernite, Y., Ferrandis, C.M., Hughes, S., Wolf, T., Guha, A., Werra, L.v. and Vries, H.d. (2023) StarCoder: may the source be with you! ArXiv. Available from http://arxiv.org/abs/2305.06161 [accessed Jan 29, 2025].
18. Li, L. and He, X. (2024) How Do Humans Write Code? Large Models Do It the Same Way Too. In: Anonymous Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing Association for Computational Linguistics. 4638–4649.
19. Munley, C., Jarmusch, A. and Chandrasekaran, S. (2024) LLM4VV: Developing LLM-Driven Testsuite for Compiler Validation. 60 1–13. Available from https://www.sciencedirect.com/science/article/pii/S0167739X24002449.
20. Patwardhan, N., Marrone, S. and Sansone, C. (2023) Transformers in the Real World: A Survey on NLP Applications. Information, 14(4).
21. Priya, R., Wang, X., Hu, Y. and Sun, Y. (2017-12) A Deep Dive into Automatic Code Generation Using Character Based Recurrent Neural Networks. In: Anonymous IEEE. 369.
22. Rahman, M.M., Watanobe, Y. and Nakamura, K. (2020) A Neural Network Based Intelligent Support Model for Program Code Completion. Scientific Programming, 2020 1..
23. Reed, S. and De Freitas, N. (2015) Neural Programmer-Interpreters. Available from http://arxiv.org/abs/1511.06279.
24. Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X.E., Adi, Y., Liu, J., Sauvestre, R., Remez, T., Rapin, J., Kozhevnikov, A., Evtimov, I., Bitton, J., Bhatt, M., Ferrer, C.C., Grattafiori, A., Xiong, W., Défossez, A., Copet, J., Azhar, F., Touvron, H., Martin, L., Usunier, N., Scialom, T. and Synnaeve, G. (2024) Code Llama: Open Foundation Models for Code. Available from http://arxiv.org/abs/2308.12950 [accessed Jan 29, 2025].
25. S. Hochreiter and J. Schmidhuber. (1997) Long Short-Term Memory. Neural computation, 9(8) 1735–1780..
26. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I. (August 2, 2017) Attention Is All You Need. In: Anonymous

Proceedings of the 31st International Conference on Neural Information Processing Systems Long Beach, California, USA: Curran Associates Inc. 6000–6010.
27. Zaremba, W., Mikolov, T., Joulin, A. and Fergus, R. (20-22 June) Learning Simple Algorithms From Examples. In: Anonymous The 33rd International Conference on Machine Learning, 2016. 48:421–429.
28. Zaremba, W. and Sutskever, I. (2014) Learning to Execute. arXiv (Cornell University), Available from https://arxiv.org/abs/1410.4615.